

PAPER TRADING ENGINE

Slice 1 Learning Notes

Exact parsing, venue decoding, the binary capture record, the append-only sink, and in-stream gap detection.

What was actually built and learned during Slice 1 — the decisions taken, the bugs found and how they were proven, and why each choice matters for a low-latency trading engine. It is a companion to the source, not a replacement: every section names the files and tests it describes. A `TODO(fuaad)` file is a planned component, not completed code.

This is deliberately a learning document, not a substitute for the source. Read it beside the files named in each section. A `TODO(fuaad)` file is a planned component, not completed code.

1. Current status at a glance

Area	Code status	Test status	Why it exists
Strong market types	implemented	passing	Prevent price, quantity and order-ID mix-ups.
Normalized <code>OrderEvent</code>	implemented	passing	One venue-independent representation of an L3 order event.
<code>InstrumentSpec</code>	implemented	passing	Holds per-market decimal scales; no hidden BTC/USD defaults.
<code>Result<T, E></code>	implemented	passing	Return an exact value or a reasoned failure without exceptions.
Exact decimal/integer parser	implemented	passing	Convert venue decimal/integer text into exact <code>int64_t</code> / <code>uint64_t</code> without touching a float.
<code>Clock</code>	implemented	passing	Testable seam over wall-clock/steady-clock access.
<code>byte_buffer</code> bounds-checked primitives	implemented	passing	Reserved for a future legacy-record decode path; not yet on the hot path.
Bitstamp decoder (event + chain)	implemented	passing	JSON line to <code>OrderEvent</code> , plus event-chain ids for gap detection.
<code>Record</code> (binary capture format), v2	implemented	passing	Fixed-size, versioned, gap-aware on-disk layout.
<code>Sink</code>	implemented	passing	RAII append-only binary writer.
<code>Recorder</code> (<code>runRecorder</code>)	implemented	passing	Testable capture loop: decode, build a record, gap-check, write.
Book, replay, simulator, strategy, live adapter	placeholder only	placeholder only	Slice 2 onward.

Test count: 80, all passing, all under ASan/UBSan, green under both GCC and Clang in CI.

Do not infer implementation from a filename. A header containing only a `TODO` comment is an intentional placeholder and has no behaviour yet — still true for `book/`, `engine/`, `venue/`.

2. Repository map and dependency direction

include/te/core/	small reusable, venue-independent value concepts
include/te/feed/	converts venue data into normalized engine events
include/te/telemetry/	binary capture records, the append-only sink, and the capture loop
include/te/book/	L3 order-book state (Slice 2)
include/te/engine/	event loop, strategy and portfolio (Slice 3)
include/te/venue/	simulated and later external execution (Slices 3-5)
include/te/util/	data structures and low-level helpers
tests/unit/	fast tests for one small behaviour each
tests/golden/	future recorded-input / expected-output regression tests

The desired direction is inward:

```
raw venue JSON -> feed decoder -> core types / OrderEvent -> book -> engine -> venue
```

`core` must not depend on a specific exchange, GUI, websocket, or strategy. That keeps the highest-value correctness code easy to reuse and test.

3. C++ header basics used throughout

`#pragma once`

Every current header begins with `#pragma once`. It prevents the same header being included more than once in a single translation unit. Without an include guard, repeated definitions could cause compiler errors.

`namespace te`

All engine code currently lives in namespace `te` (Trading Engine):

```
te::Price
te::Qty
te::Result<int, SomeError>
```

Namespaces prevent collisions with names from the standard library or another dependency.

Header versus source file

Headers expose declarations and small inline/template definitions. A template normally has its definition in the header because the compiler must generate a concrete type where it is used.

For example, compiling `Result<int, TestError>` requires the compiler to see all `Result<T, E>` function bodies in `result.hpp`. A generic template body hidden in a later `.cpp` file would not be available to generate that specialization.

4. Strong market types: `core/types.hpp`

Source: `include/te/core/types.hpp` Tests: `tests/unit/test_types.cpp` (10 tests)

The problem

Price and quantity are both naturally represented with integers. An alias does not protect us:

```
using Price = std::int64_t;
using Qty = std::int64_t;
```

With aliases, the compiler treats both names as the same type. Accidentally passing quantity where price belongs compiles silently -- the most dangerous kind of error in market code.

The solution: tiny wrapper structs

`Price`, `Qty`, and `OrderId` are distinct structs containing their underlying values:

Type	Stored field	Meaning
<code>Price</code>	signed <code>int64_t</code> ticks	Exact price in an instrument's smallest stored unit.
<code>Qty</code>	signed <code>int64_t</code> units	Exact quantity in the instrument's stored quantity unit.
<code>OrderId</code>	unsigned <code>uint64_t</code> value	Venue order identifier, not arithmetic quantity.
<code>Side</code>	<code>uint8_t</code> enum	<code>buy</code> or <code>sell</code> .

The `DistinctTypesDoNotInterconvert` test uses compile-time assertions to prove `Qty` cannot become `Price`, and `Price` cannot silently become an integer. This is a **design contract**, not runtime behaviour.

Exact integer pricing

For Bitstamp BTC/USD, two price decimal places mean:

```
"65168.69" -> Price{6'516'869}
```

The integer is exact. Later arithmetic adds integer ticks, which cannot accumulate binary floating-point rounding error. The price scale is not stored in every `Price`; it belongs to the venue/instrument metadata in `InstrumentSpec`.

`PriceArithmetic.TickSumsAreExact` demonstrates why production boundary code must never decode a price through `double`. The use of `double` in that test is a demonstration of the failure mode, not a production technique.

```
enum class Side : std::uint8_t
```

`enum class` is a scoped enum. Calling code must write `te::Side::buy`, rather than a loose global `buy`. Its explicit one-byte underlying type makes the intended in-memory representation clear.

Comparisons and `friend`

`Price` has defaulted equality and three-way comparison:

- `operator==` makes `first == second` compare their `ticks` members.

- `operator<=>` supplies an ordering, so `first < second`, `first <= second`, and related comparisons work.
- `= default` asks the compiler to generate memberwise comparison. That works for equality and three-way comparison, but not for a standalone `<` operator.

These are hidden non-member friends. A friend is not a member function; it receives both operands symmetrically. It could access private representation if `ticks` later becomes private. It is currently not required for access because the members are public, but preserves a clean comparison shape.

Compile-time contracts: `static_assert`

`Price`, `Qty`, `OrderId`, and `Side` have `std::is_trivially_copyable_v` assertions in their defining header. This is intentional:

```
Permanent representation requirement -> static_assert beside the type
Specific runtime result                -> unit test
```

Trivially copyable means the object has simple value-like lifetime/copy rules. It is an important precondition for later fixed-capacity queues and byte-oriented records. It does **not** by itself define a portable binary file format; field order, size, alignment, versioning, and endianness still need an explicit record design -- that record design is §13.

5. Normalized L3 event: `feed/events.hpp`

Source: `include/te/feed/events.hpp` Tests: `tests/unit/test_events.cpp` (6 tests)

The decoder translates venue-specific JSON into `OrderEvent`. The book consumes this normalized type rather than learning Bitstamp field names and JSON syntax.

Field	Meaning
<code>venue_timestamp_us</code>	Exchange timestamp in microseconds.
<code>order_id</code>	Which resting order changed.
<code>price</code>	Exact integer-tick price.
<code>quantity</code>	Venue-reported remaining/inserted quantity.
<code>side</code>	Buy or sell.
<code>kind</code>	<code>add</code> , <code>modify</code> , or <code>remove</code> .

Quantity contract

This is important enough to be written beside the field:

Event kind	Meaning of <code>quantity</code>
<code>add</code>	Quantity inserted into the book.
<code>modify</code>	New remaining quantity; it replaces the prior quantity.
<code>remove</code>	Reported deletion quantity, retained for audit only. Remove the order by ID; do not subtract this field from the price level.

The contract prevents a subtle accounting bug: deletion messages may report quantity that is not safe to treat as a book delta.

`EventKind` has a one-byte underlying-type contract. `OrderEvent` is checked as trivially copyable **after** its closing brace, because type traits require the complete type.

`sizeof(OrderEvent) == 40` is asserted alongside it
`( OrderEvent.IsTriviallyCopyableAndFixedSize )` -- this is the size `Record`, in §13, has to account for.

The test uses C++20 designated initialisers (`.price = ...`) to make a constructed event readable and prevent field-order mistakes. Later tests added

`DefaultConstructedFieldsAreZeroed` (a default-constructed `OrderEvent` must be all zero, not indeterminate, since `Record` embeds one and writes it to disk byte-for-byte) and

`EventKindValuesAreDistinct` (the three enumerators must not collide, since the decoder maps three different venue message types onto them).

6. Instrument metadata: `core/instrument.hpp`

Source: `include/te/core/instrument.hpp` Tests: `tests/unit/test_instrument.cpp` (2 tests)

An `InstrumentSpec` identifies one venue/market pair and its text scales:

```
Bitstamp BTC/USD: price_decimals = 2, quantity_decimals = 8
```

The test with a synthetic 4/3 specification proves scales are not global constants. It does not claim a real venue uses that specification.

Scale is not tick-size policy

A decimal scale tells the parser how many fractional digits to convert into an integer unit:

```
scale 2: "1.2" -> 120  
scale 8: "1.2" -> 120'000'000
```

It does not necessarily describe all venue order constraints. A venue may accept four fractional digits but permit orders only in increments of `0.0005`, and may also specify a minimum quantity or minimum notional. Those are separate rules to add only when verified from venue metadata.

7. Reasoned success/failure: `core/result.hpp`

Source: `include/te/core/result.hpp` Tests: `tests/unit/test_result.cpp` (2 tests) Decision: ADR 0003

Why this type exists

`std::optional<T>` can say "there is no result," but does not explain why. Decimal parsing needs to distinguish empty input, invalid character, excessive precision, and overflow. `Result<T, E>` holds either a success `T` or an error `E`.

```
Result<int64_t, ParseError>
    success -> exact parsed tick count
    failure -> ParseError explaining why parsing failed
```

Templates

In this declaration:

```
template <typename T, typename E>
class Result;
```

`T` and `E` are type parameters. The compiler creates a concrete specialization when used:

```
Result<int, TestError>
Result<int64_t, ParseError>
Result<OrderEvent, DecodeError>
Result<Sink, SinkError>
```

The current type is generic over the success and error types, but it is deliberately small rather than a full replacement for a standard-library `expected` type.

Storage and factories

`std::variant<T, E> storage_` holds exactly one alternative. Its public factories make the intent clear:

Call	Active value in <code>storage_</code>
<code>Result<T, E>::success(value)</code>	a <code>T</code>
<code>Result<T, E>::failure(error)</code>	an <code>E</code>

The constructors are private and `explicit`. Outside code must say `success(...)` or `failure(...)`; it cannot accidentally turn a raw integer or error enum into a `Result`.

The constructor member-initialiser list constructs `storage_` directly. This matters because assigning in the constructor body would first require a default-constructed `T`, a requirement a future successful type may not satisfy.

`std::move` does not itself move data. It permits the receiving object to move from a local parameter that will not be used again. This avoids an unnecessary copy for a potentially large future success value.

Query API

Function	Result
<code>hasValue()</code>	<code>true</code> when the variant currently holds <code>T</code> .
<code>valueIf()</code>	Pointer to <code>T</code> (or <code>const T</code>), or <code>nullptr</code> if failure.
<code>errorIf()</code>	Pointer to <code>const E</code> , or <code>nullptr</code> if success.

`std::get_if` matches this API because it returns a pointer or `nullptr`; `std::get` would assume the alternative exists and throw on the wrong alternative.

Addendum, added when `Sink` arrived: `valueIf()` was originally `const`-only. `Sink::write()` is a non-const method, so `Result<Sink, SinkError>::valueIf()` needed to return a non-const `Sink*` to be usable at all -- a `const Sink*` cannot call a non-const member function. A non-const overload was added alongside the original, verified not to break any existing `Result` test. The lesson worth keeping: a generic type's interface can look complete until a resource-owning `T` is the thing being held.

Tests

The tests cover both states:

- success has a value, value pointer is non-null, correct value is observed, error pointer is null;
- failure has no value, value pointer is null, error pointer is non-null, correct error is observed.

`ASSERT_NE(pointer, nullptr)` is deliberately used before `*pointer`: `ASSERT_*` stops the current test on failure and prevents dereferencing null. `EXPECT_*` records a failure but continues the test, which is appropriate only when later operations remain safe.

8. Test and build infrastructure

Source: `tests/CMakeLists.txt`

Each registered unit test follows this path:

```
test source -> test executable -> links te_core + GoogleTest main -> GoogleTest discovers cases -> CTest runs the
```

`te_add_test(name source)` centralises the CMake policy:

- creates an executable from one test source;
- links `te_core` and `GTest::gtest_main`;
- applies project warnings and sanitizers;
- asks GoogleTest/CMake to discover individual `TEST(Suite, Name)` cases.

Currently registered test targets, with their test counts:

<code>test_types</code>	10	<code>test_decimal</code>	25
<code>test_events</code>	6	<code>test_time</code>	4
<code>test_instrument</code>	2	<code>test_byte_buffer</code>	9
<code>test_result</code>	2	<code>test_decoder</code>	1
<code>test_record</code>	1	<code>test_record_roundtrip</code>	8
<code>test_recorder</code>	12		

That is 80 tests across 11 targets. `test_recorder` (§15) is the largest single file, because it covers the loop that ties every other component together, including the golden round-trip test.

Useful commands from the project root:

```
cmake -S . -B build
cmake --build build --target test_recorder
ctest --test-dir build --output-on-failure -R '^Recorder\.'
```

The final command runs every discovered test. The `-R` form filters by the GoogleTest suite name. CI runs the full suite twice per push -- once under GCC, once under Clang -- because the two compilers have caught different real bugs from each other this slice (§13 has the story).

9. Exact decimal and integer parsing: `core/text_to_int.hpp`

Source: `include/te/core/text_to_int.hpp` / `src/core/text_to_int.cpp` Tests: `tests/unit/test_decimal.cpp` (17 `Decimal` + 8 `ParseInteger` = 25 tests)

Why no floating point

Write the answer in one paragraph before trusting any code here: `0.1 + 0.2 != 0.3` in IEEE 754 binary floating point, because most decimal fractions have no exact binary representation. A price path that touches `double` even once inherits that error, and it does not cancel out -- it accumulates over millions of updates, which is exactly what a trading engine's price/PnL path does. `Price` and `Qty` being `int64_t` ticks is the type-level half of ADR 0004; `parseDecimal` is the parsing-level half, because the venue sends price as *text* (`"58356.10"`), and the only way to reach `5,835,610` without ever constructing a `double` is to parse the digit string directly.

`parseDecimal(text, scale) -> Result<int64_t, ParseError>`

```
"58356.10", scale 2    -> 5'835'610
"0.00171371", scale 8  -> 171'371
"1.234", scale 2       -> ParseError::excess_precision
```

`parseInteger(text) -> Result<uint64_t, ParseError>`

Used for `id_str` and `microtimestamp`, which the venue also sends as text specifically so a generic JSON pipeline cannot silently coerce a 16-digit order id through a `double` and lose precision above 2^{53} .

Overflow: check before the operation, not after

The real bug this caught: an early version checked for overflow *after* multiplying/adding, on the theory that a wrapped result would look obviously wrong. Demonstrated false under UBSan with real values -- a wrapped `int64_t` multiplication can land back *under* the overflow threshold, so a post-hoc check passes on genuinely corrupt data. The fix checks before every multiply and before every add: "will this operation overflow" computed from the operands, never "did it." This is the general shape of safe integer arithmetic, not specific to this parser.

10. Clock: core/time.hpp

Source: `include/te/core/time.hpp` / `src/core/time.cpp` Tests: `tests/unit/test_time.cpp` (4 tests)

Why a struct of callables, not a virtual base class

```
using Nanos = std::chrono::nanoseconds;
struct Clock {
    std::function<Nanos()> now;
    std::function<Nanos()> steadyNow;
};
```

No other type in this codebase uses virtual dispatch (`Result`, `Price`, `OrderEvent`, `Record` are all plain value types). A struct of two callables gives the same test seam a `virtual Nanos now() const = 0` interface would, with no vtable, and it fits the value-type style used everywhere else. `makeSystemClock()` wraps `std::chrono::system_clock / steady_clock`. Tests inject a fixed lambda instead of the real clock -- this is what makes `buildRecord`'s output deterministic in `test_record.cpp` and `test_recorder.cpp` despite depending on "the current time."

11. Bounds-checked bytes: `util/byte_buffer.hpp`

Source: `include/te/util/byte_buffer.hpp` / `src/util/byte_buffer.cpp` Tests: `tests/unit/test_byte_buffer.cpp` (9 tests)

Why it exists now but is not on the hot path yet

`writeU8/readU8/writeU64/readU64` operate on a `std::span<std::byte>` at an explicit offset. Nothing currently calls them: `Sink` writes a whole `Record` with one `memcpy` (§14), because the *current* record version is trivially copyable and that is well-defined. `byte_buffer` is reserved for the day the record format changes and an old version needs explicit, offset-by-offset decoding instead of `memcpy`-and-hope. This is the "fast-path now, legacy-path reserved" split, a deliberate choice over making every read go through explicit offsets from day one.

The bounds check has to guard two failure directions at once

A naive check (`offset + width <= buffer.size()`) can itself overflow if `offset` is near `SIZE_MAX`, producing a false pass. A naive underflow guard has the mirror problem. The real implementation checks both directions before touching memory, not one.

`memcpy`, not cast-and-assign, for the 64-bit functions

An early version wrote/read `uint64_t` by casting the byte pointer and dereferencing it, which truncated to a single byte in practice -- caught by testing against real hex values (`0xAABBCCDDEEFF0011ULL`), not by reading the code. `memcpy` between the raw bytes and the integer is the correct, alignment-safe way to reinterpret bytes as a value in C++, and is the same technique `Sink::write` uses for a whole `Record` (§14) and, from the other direction, `std::memset` uses to clear one (§13).

12. Decoding one Bitstamp line: `feed/bitstamp_decoder.hpp`

Source: `include/te/feed/bitstamp_decoder.hpp` / `src/feed/bitstamp_decoder.cpp` Tests: `tests/unit/test_decoder.cpp` (1 test, against a full real captured line) Decisions: ADR 0004 (exact decimal), ADR 0005 (order id), ADR 0006 (sequence gaps), ADR 0010 (venue)

simdjson On Demand, not a full DOM parse

On Demand reads fields lazily as they are asked for, rather than building a full parsed document tree per message. Building a DOM per message is exactly the trap the plan's own reading list warns about -- it means allocating on every single market event, which this decoder never does.

Reading price/quantity as text, not as the JSON numbers sent alongside them

Bitstamp sends both `price` (a bare JSON number) and `price_str` (the same value as text) in every message. The decoder reads only the `_str` fields. A generic JSON parser's `get_double()` on the bare number already round-trips through a binary float before your code ever sees it -- silently reintroducing the exact rounding error `int64_t` ticks exist to prevent (§9). `order_type` is the one field read as a bare integer, because Bitstamp sends no string form of it; `0` maps to `Side::buy`, `1` to `Side::sell`.

`decodeBitstampChain`: the event-chain ids, kept separate from `decodeBitstampEvent`

Bitstamp does not number its messages; each one names `event_id` (itself) and `pre_event_id` (the message that should have preceded it). `ChainLink` stores both as `std::array<char, 36>`, not `std::string_view` -- the views simdjson hands back point into a parse buffer that is destroyed when the function returns, so a `string_view` escaping the function would dangle. This is a second, independent parse of the same line rather than a change to `decodeBitstampEvent`'s existing contract, which keeps that function and its one test untouched. §16 covers what the chain is for.

13. The binary capture record: `telemetry/record.hpp` (v1 -> v2)

Source: `include/te/telemetry/record.hpp` / `src/telemetry/record.cpp` Tests: `tests/unit/test_record.cpp`, and the golden test in `tests/unit/test_recorder.cpp` Decision: ADR 0006 (the v2 change, gap markers)

Why a fixed 56 bytes, not "however big each event naturally is"

The simpler alternative is real: append each decoded event to a file some more natural way -- as another JSON line, say. Fixed size is chosen over that on purpose, for two properties variable-size formats cannot offer:

- **"Seek to record N" becomes arithmetic.** `offset = N * 56`. No scanning, no index file to keep in sync. Slice 3's replay engine and Slice 4's queue-position work both need to walk large captures, potentially by time window, without re-reading everything before the window.
- **Reading is a `memcpy`, not a parse.** Because `Record` is trivially copyable, decoding one back is a raw byte copy directly into a `Record`, with no tokenizer or allocator, on every single read.

A free consequence: `file_size == record_count * 56` is a correctness check that needs no parsing at all -- exactly what `Recorder.GoldenCaptureRoundTripsByteExact` asserts.

This mirrors a real, named split in the finance industry specifically: FIX (Financial Information eXchange), the standard exchange messaging protocol, is variable-length tag=value text, and it is slow to parse at scale for the reasons above -- which is exactly why SBE (Simple Binary Encoding) and FAST exist: fixed binary layouts per message type, offsets known in advance, effectively zero-copy reads. `Record` is a small, bespoke version of that same idea. Commercial tick-data vendors (Databento, Tardis) work the same way for the same reasons. Kafka takes a related but different path worth knowing: its messages *are* variable-length, but it builds a separate sparse offset-index file per log segment to recover fast seeking without forcing uniform message size -- the "fixed size, no index" versus "variable size, plus an index" tradeoff, and this project took the simpler side of it because 56 bytes gives no compression-driven reason to want variable sizing.

The byte layout (v2, current)

Offset	Bytes	Field	Notes
0-7	8	<code>orderEvent.venue_timestamp_us</code>	
8-15	8	<code>orderEvent.order_id</code>	
16-23	8	<code>orderEvent.price.ticks</code>	e.g. <code>5'835'610</code> = \$58,356.10
24-31	8	<code>orderEvent.quantity.units</code>	
32	1	<code>orderEvent.side</code>	
33	1	<code>orderEvent.kind</code> (<code>EventKind</code>)	
34-39	6	padding	zeroed, unused
40	1	<code>version</code>	currently <code>2</code>
41	1	<code>kind</code> (<code>RecordKind</code>)	added in v2; see below
42-47	6	padding	zeroed, unused
48-55	8	<code>receipt_timestamp_us</code>	

Every multi-byte field is a 64-bit integer, which the compiler must 8-byte-align -- that alignment requirement is what produces the padding, not deliberate slack left for later. `static_assert`

`(sizeof(Record) == 56)` and `static_assert(std::is_trivially_copyable_v<Record>)` sit directly below the struct, pinning both properties this whole design depends on.

Why padding had to be explicitly zeroed: `Record record{}` was not enough

`Record` is written to disk byte-for-byte, so its padding bytes end up in the capture file. They originally held whatever was already on the stack. Two attempts before the one that worked, each verified by actually building and running, not by reading the code:

Attempt	What it zeroes	Verified result
<code>Record record;</code>	Only members with their own default initialiser	Padding: stack garbage
<code>Record record{};</code>	Every named member, recursively	Still leaves padding untouched -- padding is not a "member" the initialisation rules reach
<code>std::memset(&record, 0, sizeof(record))</code>	Every byte in the object, named or not	0 of 1,433 records had non-zero padding

Impact was narrower than it first looked: a given binary produced *stable* output run to run, but two *different builds* produced different padding for identical input, because code layout changes what happens to be sitting on the stack. That breaks byte-comparing or checksumming capture files across builds, which matters for a project whose deliverable is reproducibility. This is the same bug class Valgrind's `memcheck` and Clang's `MemorySanitizer` exist to catch generally -- reading or persisting uninitialised memory -- and the same property the Reproducible Builds movement (Debian, Bazel) targets at a much larger scale: identical source and input should produce byte-identical output.

The GCC wrinkle: `-Wclass-memaccess`

`std::memset(&record, 0, sizeof(record))` compiled cleanly under Clang and broke CI under GCC:

```
error: 'void* memset(void*, int, size_t)' clearing an object of non-trivial type
      'struct te::Record'; use assignment or value-initialization instead
[-Werror=class-memaccess]
```

`OrderEvent`'s default member initialisers make `Record` non-trivially-*default-constructible* (distinct from trivially-copyable, which is what actually matters here and is what the `static_assert` checks). GCC cannot statically prove `memset`-ing an arbitrary class is safe -- it would be genuinely dangerous on a type with a vtable or an owning pointer -- so it warns generically. The fix is `std::memset(static_cast<void*>(&record), 0, sizeof(record))`, the conventional way to tell the compiler this is a deliberate raw-memory operation. Local development is Clang-only, which is why this only surfaced once CI ran GCC -- the direct reason CI now builds both compilers rather than one.

v1 -> v2: adding `RecordKind` for free

Adding `RecordKind kind` (offset 41, §16) bumped `kCurrentRecordVersion` from 1 to 2, but `sizeof(Record)` stayed 56, because the new byte landed inside what was already padding. Better still, **v1 captures need no converter**: `RecordKind::order_event == 0`, and v1 already zeroed its padding (the fix above), so a v1 record's byte 41 already reads as `order_event` under v2 rules. Same byte, same value, only the meaning changed -- and that only holds because padding was zeroed first.

14. Sink: telemetry/sink.hpp

Source: `include/te/telemetry/sink.hpp` / `src/telemetry/sink.cpp` Tests: Sink-specific cases in `tests/unit/test_record_roundtrip.cpp`

Why a dedicated type instead of a raw `std::ofstream`

`Sink` wraps exactly one `std::ofstream`. The alternative -- just using `ofstream` directly at every call site -- would mean every caller independently remembers to check `is_open()`, handle failure without exceptions (this project's own convention, ADR 0002/0003), and know the flush policy. `Sink::open()` instead returns this project's own `Result<Sink, SinkError>`, so file I/O speaks the same error-handling language as decoding and parsing. A raw C `FILE*` / `fwrite` alternative loses C++ RAII entirely, requiring a manual `fclose()` on every exit path including every early `return` -- exactly the bug class RAII exists to make impossible by construction: "the file is closed automatically when the `Sink` is destroyed, so an early return cannot leak a handle."

A virtual `Writer` interface (swappable file/network/memory implementations) was deliberately *not* built. There is exactly one implementation today, and `sink.hpp`'s own header comment says why that is temporary: "Append-only file sink now, ZeroMQ sink in slice 5." Building the abstraction before a second implementation exists to justify it would be paying a real cost (an indirect call per write) for a benefit -- swappability -- that does not exist yet.

"Sink" itself borrows established vocabulary rather than inventing project-local naming: it is the term `spdlog` and `Serilog` use for the destination side of a log stream, and the same word Kafka Streams and Apache Flink use for their own output connectors (`KafkaSink`, `FileSink`).

`std::ios::app` and its atomicity guarantee

Opened with `std::ios::app`, which on POSIX (macOS, Linux) maps to the kernel's `O_APPEND` flag. The property that matters is not "writes land at the end" -- that alone could be done by manually seeking first -- it is that `O_APPEND` makes *seek-to-end-then-write* a single atomic kernel operation. Without it, two concurrent writers each doing "seek to end, then write" as two separate steps can race: both seek to the same offset before either has written, and one overwrites or interleaves with the other. POSIX specifies that no other file-modifying operation may occur between the offset being set and the write landing when `O_APPEND` is set. This is the same guarantee `rsyslog`, `journald`, and a database's write-ahead log all depend on when multiple writers (or a crashed-and-restarted single writer) append to one file -- it is what makes "safe to restart after a crash" true almost for free, since a relaunched recorder's writes are guaranteed to land after whatever is already there, never overlapping it.

`flush()`, and the three places a byte can be before it is safe

Layer	Where	Survives this process crashing	Survives the machine losing power
1. Stream buffer	This process's own memory	No	No
2. OS page cache	Kernel memory	Yes	No
3. Physical disk	The device itself	Yes	Yes

`sink.write(record)` only reaches layer 1 -- copied into `ofstream`'s own buffer, purely for speed (batching many small writes into one syscall; a syscall costs real time). `sink.flush()` forces a real `write()` syscall, moving the bytes to layer 2, the OS page cache -- which is why it survives *this process* dying, even though it has not reached the physical device yet. Reaching layer 3 needs `fsync()`, a heavier POSIX call `std::ofstream` does not expose at all, which is why `sink.hpp`'s own doc comment is explicit that `flush()` "is not the same as durability."

`runRecorder` (§15) flushes after every record, which makes the loss window on an unclean shutdown of *this process* exactly zero records; a whole-machine power loss can still lose whatever was flushed but not yet `fsync` ed. This exact tradeoff is a named, load-bearing decision in real storage systems: Redis's `appendfsync` setting exposes it directly (`always` / `everysec` / `no`); Postgres's `fsync` setting is the same knob; Kafka notably chose *not* to `fsync` per message at all, relying on the OS page cache plus cross-broker replication for durability instead, trading single-node durability for throughput.

15. The capture loop: `telemetry/recorder.hpp` + `apps/recorder_main.cpp`

Source: `include/te/telemetry/recorder.hpp` / `src/telemetry/recorder.cpp` Tests: `tests/unit/test_recorder.cpp` (12 tests)

Why the loop takes streams, not paths

```
Result<RecorderStats, RecorderError> runRecorder(
    std::istream& input, Sink& sink, const InstrumentSpec& spec, const Clock& clock);
```

The loop originally lived entirely inside `main()`. That is untestable by construction -- there is no `main()` in a test binary, so the only verification was running the program and reading numbers off the terminal. Taking a `std::istream&` and a `Sink&` rather than paths means a test builds a `std::istringstream` from a literal string and points a real `Sink` at a temp file, then calls the *exact same function* `main()` calls -- no filesystem assumptions, no argv, no network. `main()` is now a 73-line shell: argv handling, a check that refuses to write into an existing output path (`Sink` appends by design; re-running the recorder at the same path would otherwise silently concatenate two sessions), and reporting.

Counters are the run's only observability surface

`RecorderStats{linesRead, written, skipped, failed, gapsDetected}`. `not_order_event` (a Bitstamp protocol message like `bts:subscription_succeeded`) counts as `skipped`, not `failed` -- it is present in every healthy capture, and counting it as an error would make a clean run look broken, which trains you to stop reading the error count. The loop asserts `linesRead == (written - gapsDetected) + skipped + failed`, computed two independent ways (once per input line, once per branch taken), specifically so a line that took an unaccounted path is caught rather than silently dropped. This is what the golden test (`Recorder.GoldenCaptureRoundTripsByteExact`) checks against the real 1,434-line capture: 1,433 written, 1 skipped, 0 failed, output exactly `1433 * 56 = 80,248` bytes, zero non-zero padding across every record.

16. Gap detection: ADR 0006, kept inside the stream

Source: chain extraction in `bitstamp_decoder.cpp` (§12), tracking in `recorder.cpp` (§15), the marker itself in `record.hpp` (§13)

A chain, not a sequence number

Bitstamp does not number messages. Each one names `event_id` (itself) and `pre_event_id` (the id of the message that should have preceded it), so continuity is a *link* check -- does this message's `pre_event_id` match the previous message's `event_id` -- not `expected == actual + 1`. A broken link proves data was lost, with no exact count to compare against.

Refinement, from ADR 0006's own Amendment 1, not originally caught here: the

`event_id`'s final UUID segment turns out to be a dense four-state counter, so lost-event count is recoverable *modulo 4* -- not exact, and not enough to tell a 1-event drop from a 4n+1-event burst apart, which is why the actual policy (any break fully invalidates the book) is unchanged. "No magnitude whatsoever" was stated flatly in an earlier pass of this file; that overstates it.

Why the marker lives in the stream, not a sidecar file

A gap could instead be recorded in a separate file listing breaks. That makes correctness *opt-in*: the one replay that forgets to read the sidecar produces a confidently wrong result with no visible symptom. Putting a `RecordKind::gap` marker inside the record stream itself, at the exact position where continuity broke, means a reader cannot miss it without deliberately ignoring record kinds. Kafka's control records and a database's WAL checkpoint records take the same in-stream approach.

Three cases that must not produce a false gap

Case	Why it must not count
The first order event in a capture	Has no predecessor. Without an explicit guard, every capture would open with a false gap.
Protocol messages (<code>bts:subscription_succeeded</code>)	Carry no chain ids at all.
An unreadable chain	Absence of evidence is not evidence of loss; inventing gaps would censor good sessions.

All three are covered by dedicated tests in `test_recorder.cpp`

(`FirstEventCannotBreakTheChain`, `SubscriptionMessageDoesNotBreakTheChain`, and the chain-hasValue guard exercised in the main gap test).

The real capture's chain is intact

Across all 1,432 links in the 1,434-line reference capture, zero breaks -- so gap *detection* is verified against synthetic fixture lines (including a deliberately dangling `pre_event_id` built from the classic `deadbeef` hex value, chosen because it can never collide with a real UUID), not against the one real capture currently on disk.

17. Placeholder file map: later, not today

Directory	Later responsibility
<code>book/*</code>	L3 order book and FIFO price levels, Slice 2. ADR 0007 (price-level storage) is open; see <code>docs/coding-plan-v3.md</code> §4. Note: <code>scripts/dump_raw_ws_bitstamp.py</code> already captures a paired <code>.snapshot</code> file (REST <code>group=2</code>) alongside every <code>.jsonl</code> segment -- the missing piece is a C++ parser/seed step to consume it, not a new capture mechanism.
<code>engine/*</code> , <code>feed/feed.hpp</code> , <code>feed/replay_feed.hpp</code> , <code>venue/simulated_venue.hpp</code> , <code>venue/venue.hpp</code>	Offline replay and simulated execution, Slice 3.
<code>venue/queue_model.hpp</code> , <code>venue/latency_model.hpp</code>	Queue/fill model, Slice 4.
<code>feed/live_feed.hpp</code> , <code>venue/coinbase_venue.hpp</code>	Live observation/execution boundaries, Slice 5. Current plan treats external execution as optional.
<code>util/spsc_queue.hpp</code>	Feed-thread-to-strategy-thread queue, Slice 3.
<code>util/memory_pool.hpp</code> , <code>util/intrusive_list.hpp</code>	Fixed-block allocator and self-linking order nodes for the book, Slice 2.

`byte_buffer.hpp`, `record.hpp`, `sink.hpp`, and `bitstamp_decoder.hpp` have moved out of this table since the last revision of this document -- all four are implemented and tested (§§11-14).

18. Resume checklist

1. Slice 1's capture path is complete: decode, build a record, detect gaps, write, verified end-to-end against the real capture (§§9-16). The websocket client is deliberately deferred; the file-driven path is the whole recorder for now.
2. Read `docs/coding-plan-v3.md` §4 and `docs/decisions/0007-book-level-storage.md` before writing any Slice 2 code. Both were corrected against `scripts/dump_raw_ws_bitstamp.py` and the real `.snapshot` file it already produces (4,174 bids / 4,563 asks, `group=2`) -- the venue's own `live_orders` channel sends no initial snapshot over the websocket, but this project's Python capture layer already fetches one over REST per ADR 0006. The open work is a C++ consumer for it, not a new capture mechanism.
3. Decide ADR 0007 (price-level storage) and write down the actual band width and exit policy, not just which storage shape.
4. `OrderBook`, `Pool<T>`, and the intrusive list bodies are yours to write, per this project's founding rule -- the plan document gives declarations and invariants, not bodies.

19. Quick self-check

1. Why are `Price` and `Qty` structs instead of `using` aliases for `int64_t`?
2. What does two-decimal scale do to the text `"1.2"`?
3. Why is a `static_assert` beside `OrderEvent` stronger than an equivalent assertion in a test?
4. In `Result<int64_t, ParseError>`, what does `valueIf()` return after `failure(...)`?
5. Why does `Result<T, E>` live fully in a header?
6. Why must `ASSERT_NE(pointer, nullptr)` occur before dereferencing a test pointer?
7. Why does `parseDecimal` check for overflow *before* each multiply/add rather than after?
8. `Record record{};` zeroes every named member. Why did the capture file still contain non-zero bytes after that change?
9. What does `sink.flush()` actually guarantee, and what does it not guarantee?
10. Why does `O_APPEND` matter for a file `Sink` opens, even with only one writer today?
11. `sizeof(Record)` did not change between v1 and v2. How was a new field added without growing it?
12. Why is a broken event chain detected but never sized -- why can't you know how many messages were lost?

Answers: (1) compile-time domain safety; (2) `120` ticks; (3) it enforces a permanent contract in every build at the defining location; (4) `nullptr`; (5) the compiler needs its definitions to generate each specialization; (6) it stops an unsafe null dereference after failure; (7) a wrapped `int64_t` result can land back under the overflow threshold, so a check performed after the operation can pass on already-corrupt data; (8) value-initialisation zeroes named members, not the compiler-inserted padding between them -- only `std::memset` over the whole object reaches padding; (9) it guarantees the bytes reached the OS page cache, so a crash of this process cannot lose them; it does not guarantee they reached physical storage, so a whole-machine power loss still can; (10) it makes seek-to-end-then-write atomic, which is what lets a crashed-and-restarted writer resume without a race against its own prior writes; (11) the new field was placed inside bytes that were already alignment padding; (12) Bitstamp chains messages by id (`pre_event_id` naming the prior `event_id`) rather than numbering them, so a break proves loss without any count to compare against.